

SOFTWARE CONSTRUCTION RECORD

NAME: Priyan S, Phaveen S,
Rajiv Gandhi S

REGNO:240701348,24070138
3,240701414

EXPERIMENT – 1

STUDY OF AZURE

Aim:

To study the Azure, benefits, the services provided by Azure.

What is Azure?

Microsoft Azure is a cloud computing platform developed by Microsoft. It provides on-demand access to computing resources like virtual machines, databases, storage, and networking through the internet.

Azure helps individuals and organizations build, deploy, and manage applications without needing physical hardware, using a global network of data centers.

Benefits of Azure

- **Cost-effective** – Pay only for what you use (no upfront hardware cost)
- **Scalability** – Easily increase or decrease resources based on demand
- **High availability** – Reliable uptime with global data centers
- **Security** – Advanced security features and compliance certifications
- **Integration** – Works well with Microsoft tools like Windows Server and Microsoft 365
- **Flexibility** – Supports multiple programming languages and frameworks

Services Provided

1. Infrastructure as a Service (IaaS)

Provides basic infrastructure like virtual machines and storage.

- Virtual Machines
- Storage (Blob, Disk)
- Networking
- Users manage OS, applications, and data.

2. Platform as a Service (PaaS)

Provides a platform to develop and deploy applications without managing infrastructure.

- Azure App Services
- Azure SQL Database
- Azure Functions
- Azure manages OS, runtime, and infrastructure.

3. Software as a Service (SaaS)

Provides ready-to-use software over the internet.

- Microsoft 365
- Dynamics 365
- Users just use the application—no management needed.

Web Services (Functionality in Azure)

Azure offers web-based services to build and host applications:

- **Hosting Web Apps** – Using Azure App Service
- **Serverless Computing** – With Azure Functions
- **API Management** – Create and manage APIs
- **Content Delivery Network (CDN)** – Fast content delivery worldwide
- **Load Balancing** – Distributes traffic efficiently
- **Authentication & Security** – Secure user access
- **Database Integration** – Connect apps with cloud databases

These services help developers build scalable, secure, and high-performance web applications.

Comparison: Azure vs AWS vs Google Cloud

Feature	Microsoft Azure	Amazon Web Services (AWS)	Google Cloud Platform (GCP)
Provider	Microsoft	Amazon	Google
Market Position	Strong enterprise presence	Market leader	Growing rapidly
Best For	Microsoft ecosystem integration	Wide range of services	Data analytics & AI
Ease of Use	Moderate	Complex but powerful	Beginner-friendly
Pricing	Flexible	Complex pricing	Competitive pricing
Hybrid Cloud	Excellent	Moderate	Good
AI & ML	Strong (Azure AI)	Very strong	Very strong (Google AI)

Result:

Thus the study of Azure is done.

Aim:

To understand the problem statement and to create user stories for the problem statement based on functional and non-functional requirements.

Problem Statement:

Government Medical Colleges face challenges in managing communication and workflow between Supervisors, Therapists, and administrative staff in Speech Therapy departments. Most therapy plans, session notes, and approvals are maintained manually using paper records or informal communication methods.

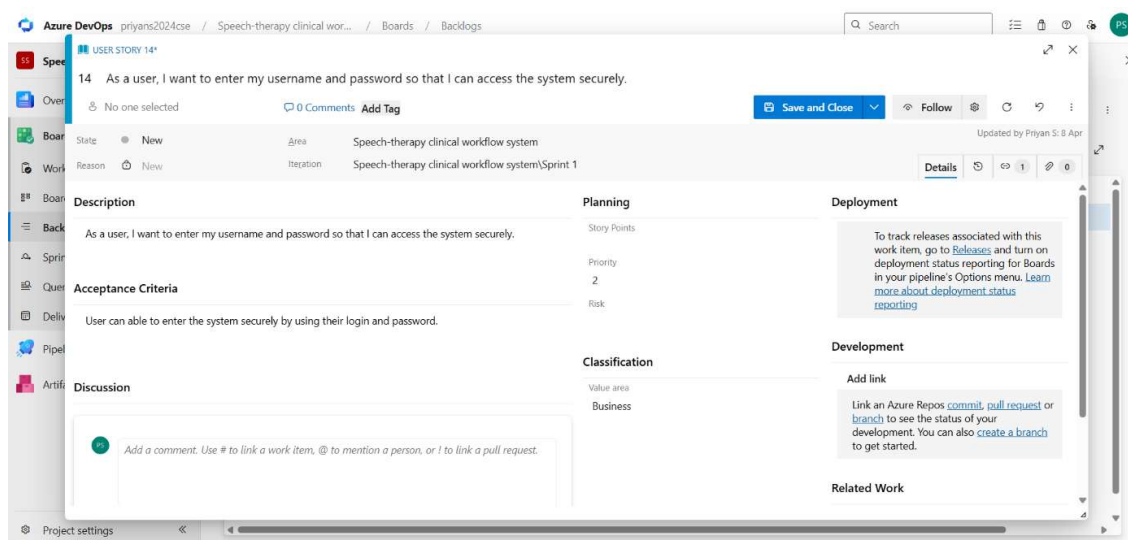
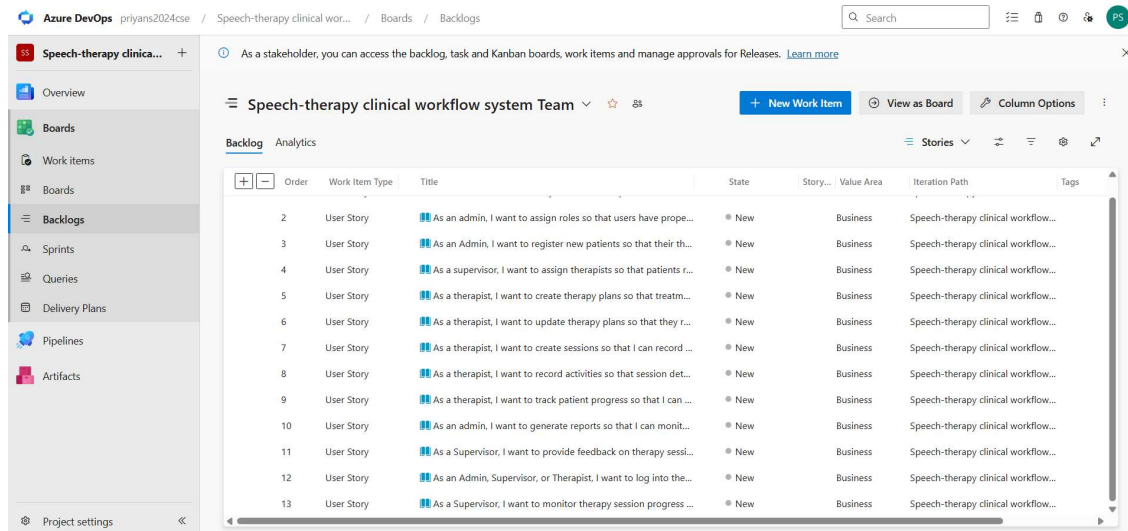
Procedure:

1. Open Azure DevOps and sign in with valid credentials.
2. Create a new Agile project named “Speech Therapy Clinical Workflow System”.
3. Go to Boards from the left-side navigation menu.
4. Select Backlogs to manage project user stories.
5. Click on New Work Item and choose User Story.
6. Enter the title of the user story using the format:
“As a user, I want..., so that...”
7. Add the description for the user story based on the project requirements.
8. Define the Acceptance Criteria for validating the functionality.
9. Set the State as “New”.
10. Assign Story Points, Priority, and Risk level if required.
11. Select the Iteration Path (Sprint) for the user story.
12. Save the user story using Save and Close option.
13. Repeat the process for all required user stories such as:
 - User Login

- Register Patient
- Assign Therapist
- Create Therapy Plan
- Update Session Notes
- Provide Feedback
- Generate Reports

14. Verify all created user stories in the Backlog section.

15. Review the user stories to ensure all functional requirements are covered properly.



Result:

Thus, the problem statement has been understood and user stories has been made based on the functional and non-functional requirements.

EXPERIMENT – 3

USE CASE DIAGRAM

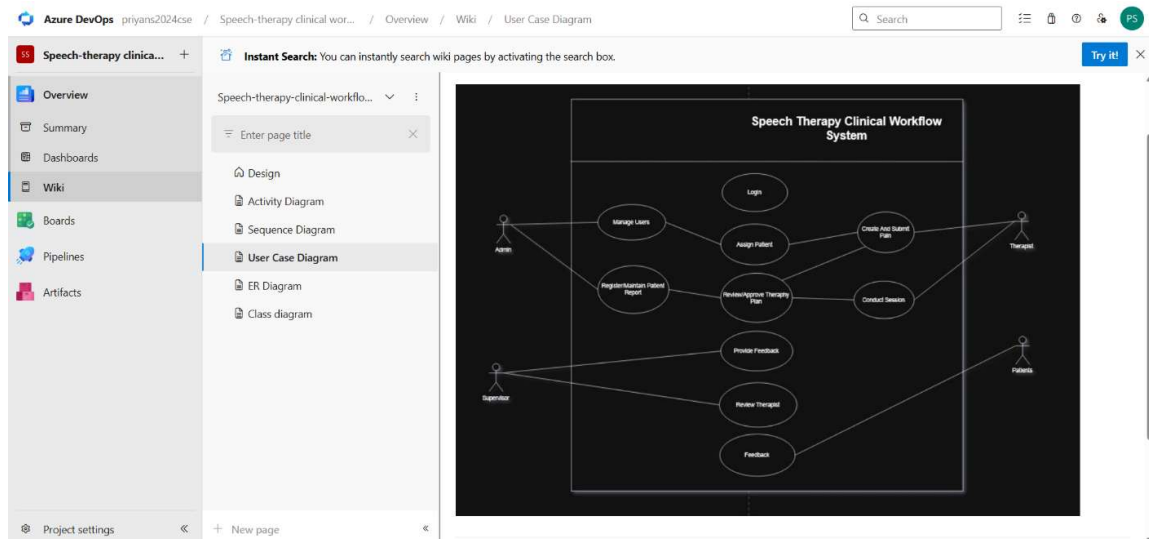
Aim:

To draw the use case diagram based on the problem statement.

Procedure:

1. Open Azure DevOps and select the project “Speech Therapy Clinical Workflow System”.
2. Go to the Wiki section from the left-side navigation menu.
3. Create a new wiki page named User Case Diagram.
4. Identify the system boundary for the Speech Therapy Clinical Workflow System.
5. Identify the actors interacting with the system such as:
 - Admin
 - Supervisor
 - Therapist
 - Patients
6. Identify the major functionalities of the system.
7. Create use cases including:
 - Login
 - Manage Users
 - Assign Patient
 - Register/Maintain Patient Report
 - Create and Submit Plan
 - Review/Approve Therapy Plan
 - Conduct Session
 - Provide Feedback
 - Review Therapist
 - Feedback

8. Draw the system boundary using a rectangle.
9. Represent actors using stick figures outside the system boundary.
10. Represent use cases using oval shapes inside the system boundary.
11. Connect actors and use cases using association lines.
12. Arrange the use cases properly for better readability and workflow understanding.
13. Save the diagram in the Wiki page.
14. Review the use case diagram to ensure all system functionalities are represented correctly.



Result:

Thus, the use case diagram has been drawn successfully.

EXPERIMENT – 4

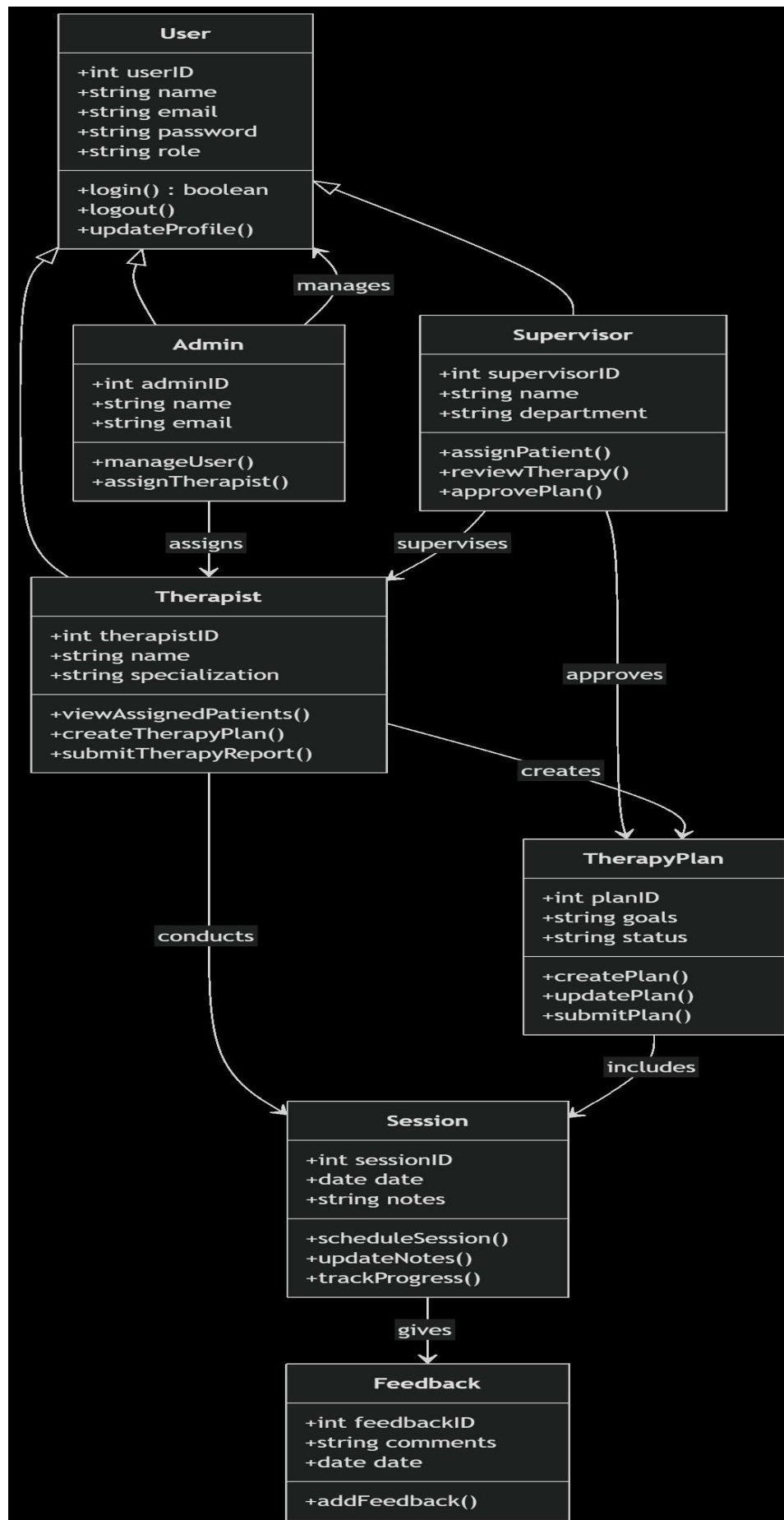
CLASS DIAGRAM

Aim:

To draw the Class Diagram for the problem statement.

Procedure:

1. Open Azure DevOps and select the project “Speech Therapy Clinical Workflow System”.
2. Go to the Wiki section and create a page named “Class Diagram”.
3. Identify the main classes such as User, Admin, Supervisor, Therapist, TherapyPlan, Session, and Feedback.
4. Add attributes and methods for each class using proper data types.
5. Draw relationships between classes such as inheritance and association.
6. Connect the classes using arrows and label the relationships.
7. Arrange the classes properly for better understanding.
8. Save the class diagram in the Wiki page and review it for correctness.



Result:

Thus, the class diagram has been drawn for the problem statement.

EXPERIMENT – 5

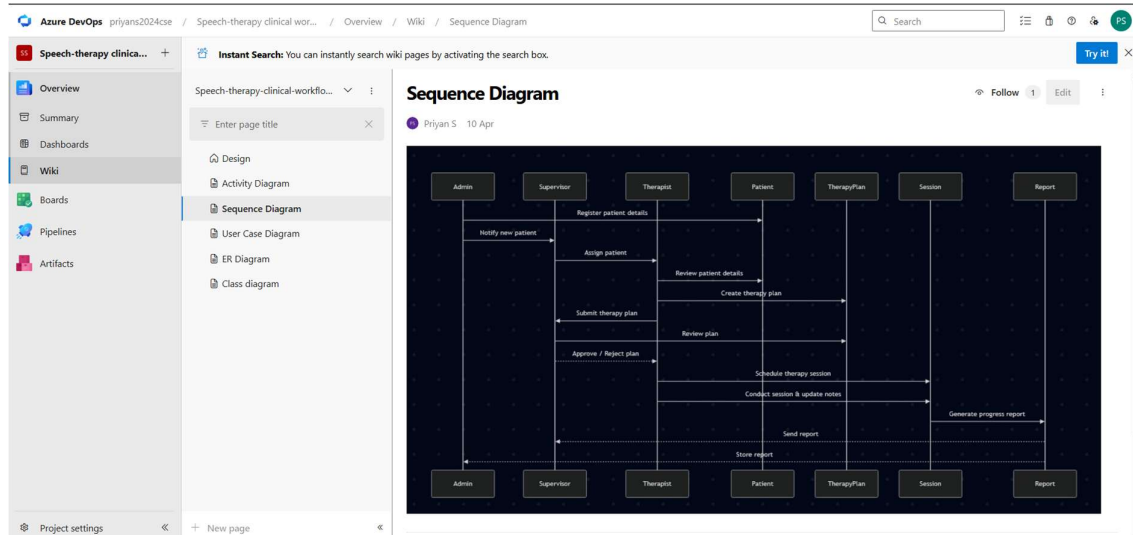
SEQUENCE DIAGRAM

Aim:

To draw sequence diagram for the problem statement.

Procedure:

1. Open Azure DevOps and select the project “Speech Therapy Clinical Workflow System”.
2. Navigate to the Wiki section and create a page named “Sequence Diagram”.
3. Identify the participants involved such as Admin, Supervisor, Therapist, Patient, TherapyPlan, Session, and Report.
4. Draw lifelines for all participants in the sequence diagram.
5. Add interactions between the participants based on the system workflow.
6. Represent messages such as patient registration, patient assignment, therapy plan creation, and therapy plan approval.
7. Add session scheduling and session update interactions in the diagram.
8. Represent report generation and report storage processes.
9. Arrange all messages in proper time sequence from top to bottom.
10. Save the sequence diagram in the Wiki page and review the interaction flow for correctness.
- 11.



Result:

Thus, the sequence diagram for the given problem statement has been drawn.

EXPERIMENT – 6

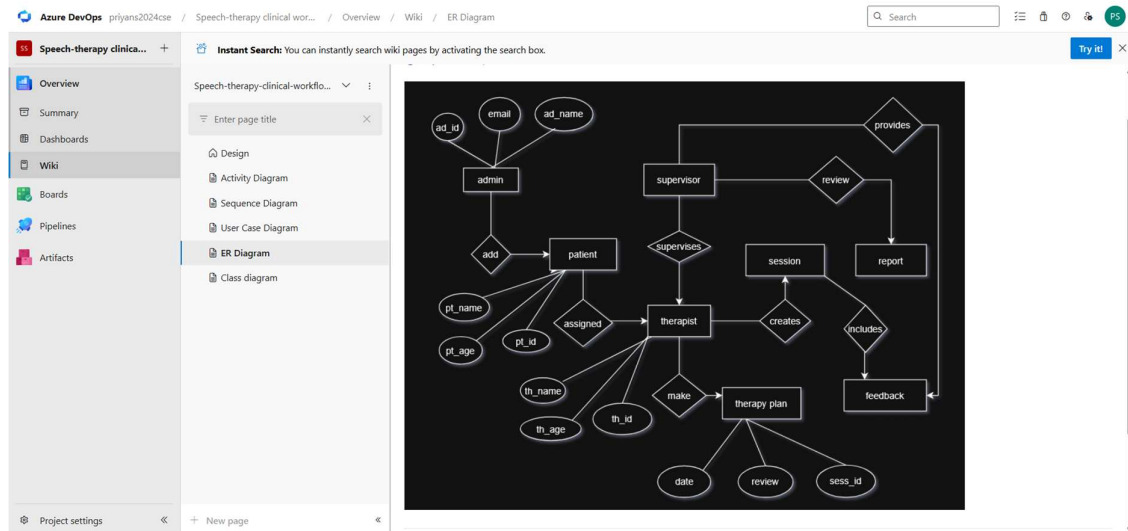
ER DIAGRAM

Aim:

To draw the Entity – Relationship diagram for the given problem statement.

Procedure:

1. Open Azure DevOps and select the project “Speech Therapy Clinical Workflow System”.
2. Go to the Wiki section and create a page named “ER Diagram”.
3. Identify the entities involved such as Admin, Supervisor, Therapist, Patient, Session, Therapy Plan, Report, and Feedback.
4. Add attributes for each entity such as ID, name, age, email, and session details.
5. Identify the relationships between entities like add, assign, supervise, create, review, and provide feedback.
6. Draw entities using rectangles and attributes using oval shapes.
7. Represent relationships using diamond shapes and connect them properly.
8. Add primary keys and important attributes for all entities.
9. Arrange all entities and relationships clearly for better understanding.
10. Save the ER diagram in the Wiki page and review it for correctness.



Result:

Thus, the ER diagram for the given problem statement is drawn.

EXPERIMENT – 7

ARCHITECTURE DIAGRAM

Aim:

To draw architecture diagram for the given problem statement.

Procedure:

1. Open Azure DevOps and select the project “Speech Therapy Clinical Workflow System”.
2. Go to the Wiki section and create a page named “Architecture Diagram”.
3. Identify the main components of the system such as Frontend, Backend, Database, and Users.
4. Include users like Admin, Supervisor, and Therapist in the architecture.
5. Represent the frontend layer for user interaction through web application interfaces.
6. Represent the backend layer for handling business logic and workflow management.
7. Add the database component for storing patient records, therapy plans, sessions, and reports.
8. Draw communication flow between users, frontend, backend, and database components.
9. Include Azure cloud services or deployment components if required.
10. Save the architecture diagram in the Wiki page and review the system structure for correctness.

Azure DevOps priyans2024cse / Speech-therapy clinical wor... / Overview / Wiki / Architecture Diagram

Search

Speech-therapy clinical wor... +

Speech-therapy-clinical-workf... ▾

Overview

Summary

Dashboards

Wiki

Boards

Repos

Pipelines

Test Plans

Artifacts

Enter page title

Design

Architecture Diagram

Activity Diagram

Sequence Diagram

User Case Diagram

ER Diagram

Class diagram

Architecture Diagram

Priyan S. Just now

Unfollow 1 Edit

```
graph TD; Admin[Admin] --- UI[Web application UI]; Superstar[Superstar] --- UI; Therapist[Therapist] --- UI; UI --- AS[Application Server Node.js / Python]; AS --- Auth[Authentication module]; AS --- PM[Patient Management module]; AS --- TP[Therapy Plan module]; AS --- SM[Session Management module]; AS --- FR[Feedback & Reporting module]; Auth --- AG[API Gateway]; PM --- AG; TP --- AG; SM --- AG; FR --- AG;
```

0 votes in last 30 days

Comments

Add a comment...

Result:

Thus, the architecture diagram for the given problem statement has been drawn.

EXPERIMENT – 8

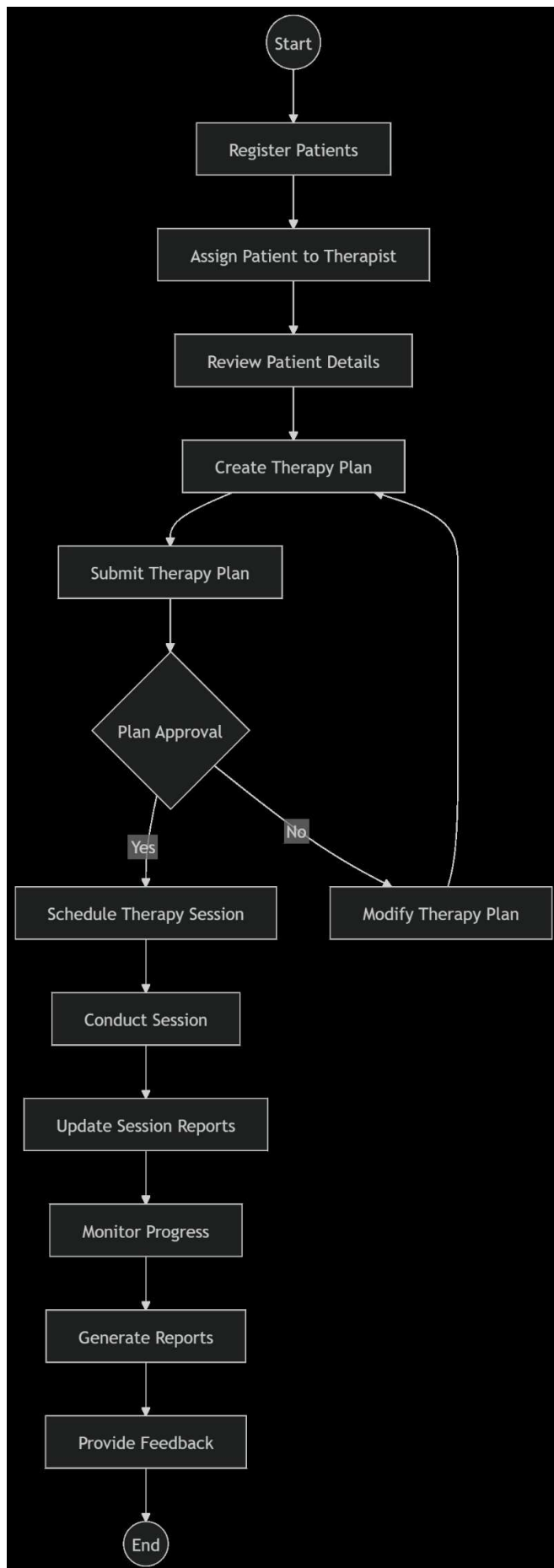
ACTIVITY DIAGRAM

Aim:

To draw the activity diagram for the given problem statement.

Procedure:

1. Open Azure DevOps and select the project “Speech Therapy Clinical Workflow System”.
2. Go to the Wiki section and create a page named “Activity Diagram”.
3. Identify the workflow involved in the therapy management process.
4. Add the start node and end node for the activity flow.
5. Represent activities such as patient registration, patient assignment, therapy plan creation, and session scheduling.
6. Add decision nodes for therapy plan approval and modification process.
7. Connect all activities using arrows to represent workflow sequence.
8. Include activities like conducting sessions, updating reports, monitoring progress, and providing feedback.
9. Arrange all activities properly for better readability and understanding.
10. Save the activity diagram in the Wiki page and review the workflow for correctness.



Result:

Thus, the activity diagram for the given problem statement has been drawn.

EXPERIMENT – 9

AZURE PROJECT FILES AND PIPELINE

Aim:

To run pipeline for the project.

Procedure:

1. Open Azure DevOps and select the project “Speech Therapy Clinical Workflow System”.
2. Go to the Pipelines section from the left-side navigation menu.
3. Click on “Create Pipeline” to configure the project pipeline.
4. Select the source code repository for the project.
5. Choose the appropriate pipeline configuration method.
6. Configure the build and testing steps for the application.
7. Add necessary project files and environment settings to the pipeline.
8. Save the pipeline configuration and run the pipeline.
9. Monitor the build and execution status in the pipeline dashboard.
10. Verify whether the pipeline executed successfully without errors.

Azure DevOps priyans2024cse / Speech-therapy clinical wor... / Pipelines / Speech-therapy clinical wor... / 20260430.1

Speech-therapy clinical workflow system

#20260430.1 - Set up CI with Azure Pipelines

Speech-therapy clinical workflow system

This run is being retained as one of 3 recent runs by main (branch).

View retention leases

Summary Code Coverage

Individual CI by Priyan S

Apr 30 at 8:51 PM 34s

Repository and version

Speech-therapy clinical workflow system

main #421a4d

Related

2 work items

0 published

Tests and coverage

Get started

View 14 changes Parameters

Warnings 2

Task 'Node.js tool installer' version 0 (NodeTool@0) is deprecated.

initiate job

This task is deprecated and will no longer receive updates. Please use UseNodeV1 as a replacement.

initiate job

Stages Jobs

Stage 0

1 job completed 33s

Job 30s

Rerun Stage

Project settings

Azure DevOps priyans2024cse / Speech-therapy clinical wor... / Pipelines / Speech-therapy clinical wor...

Speech-therapy clinical workflow system

Runs Stages Branches Analytics

Description	Stages	
#20260430.1 - Set up CI with Azure Pipelines		19 Apr 30 34s
Individual CI by Priyan S		

Result:

Thus , the pipeline was executed and run successfully.

Aim:

To create and execute a test plan in Azure DevOps for validating application functionalities.

Procedure:

1. Open Azure DevOps and select the project “Speech Therapy Clinical Workflow System”.
2. Go to the Test Plans section from the left-side navigation menu.
3. Create a new Test Plan for the project.
4. Enter the test plan name and select the iteration path.
5. Create a new Test Suite for organizing the test cases.
6. Add test cases for major functionalities of the system.
7. Create test cases for user login and authentication.
8. Add test cases for patient registration and patient assignment.
9. Create test cases for therapy plan creation and approval process.
10. Add test cases for session scheduling and session updates.
11. Create test cases for report generation and feedback management.
12. Define the expected result for each test case.
13. Execute all the test cases manually in the Execute section.
14. Mark the test cases as Pass or Fail based on the output.
15. Record defects or errors if any issues are identified.
16. Review the test results and verify the system functionality.

Azure DevOps | priyans2024cse / Speech-therapy clinical wor... / Test Plans / speechtherapysuit

Search

Speech-therapy clinical wor... +

Overview

Boards

Repos

Pipelines

Test Plans

Test plans

Progress report

Parameters

Configurations

Runs

Exploratory sessions

Artifacts

speechtherapysuit

May 1 - May 8 | **Pass**

100% run, 100% passed. [View report](#)

Test Suites

Filter suites by name

speechtherapysuit

Valid Login (1)

Valid Login (ID: 39)

Define | **Execute** | Chart

Test Points (1 item)

Run for web application

Title	Outcome	Order	Test Case ID	Status	Configuration	Current Tester
Valid login	Passed	1	40	Design	Windows 10	Priyan S

Result:

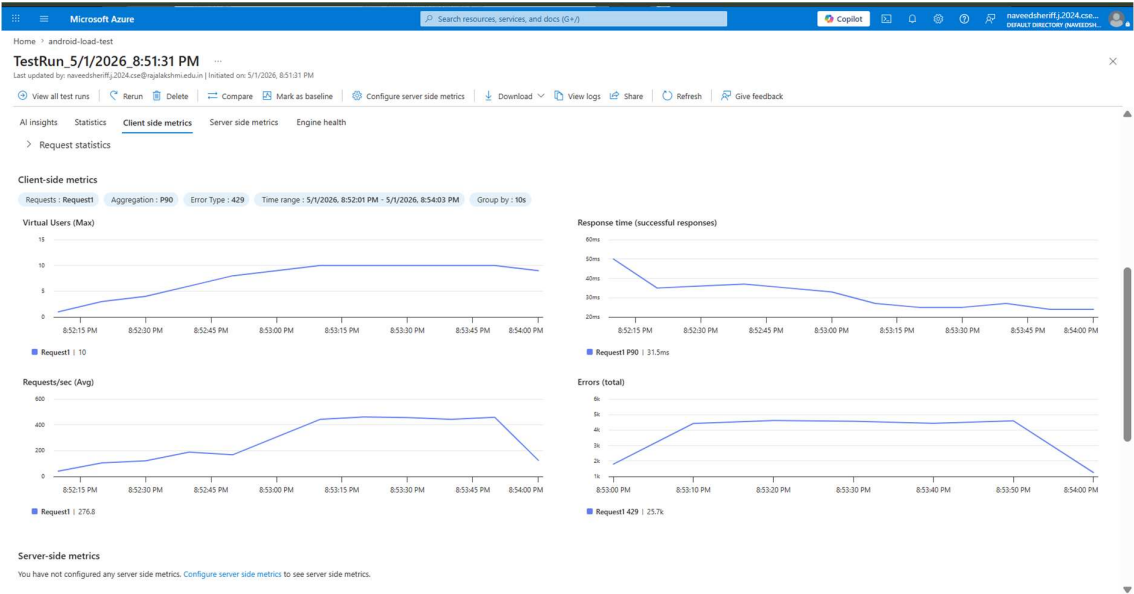
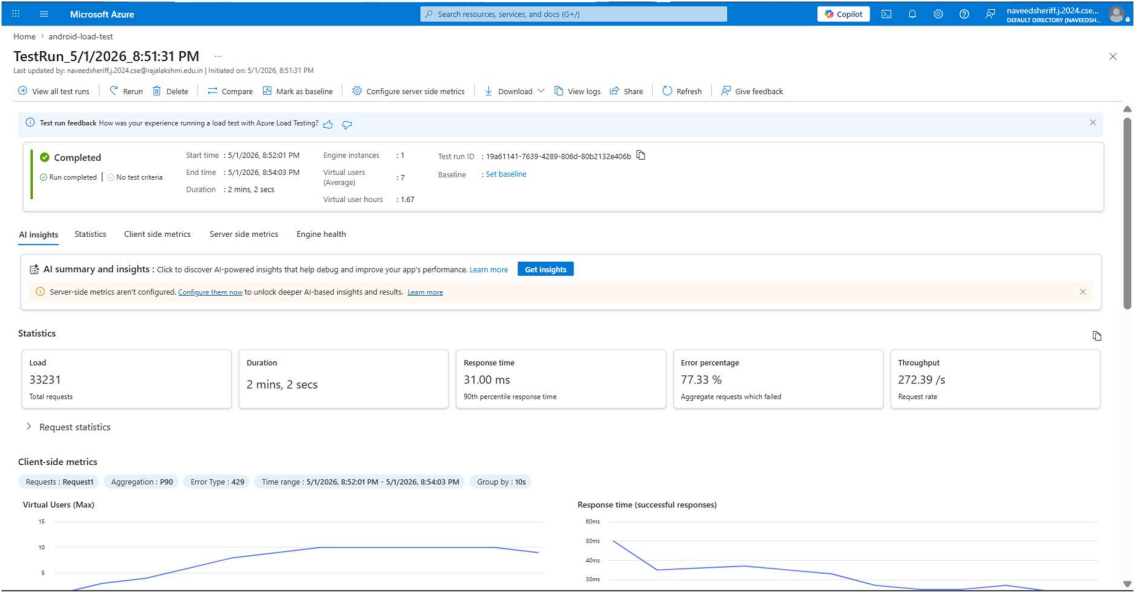
Thus, Test plan was successfully created and all test cases were executed with expected outcomes.

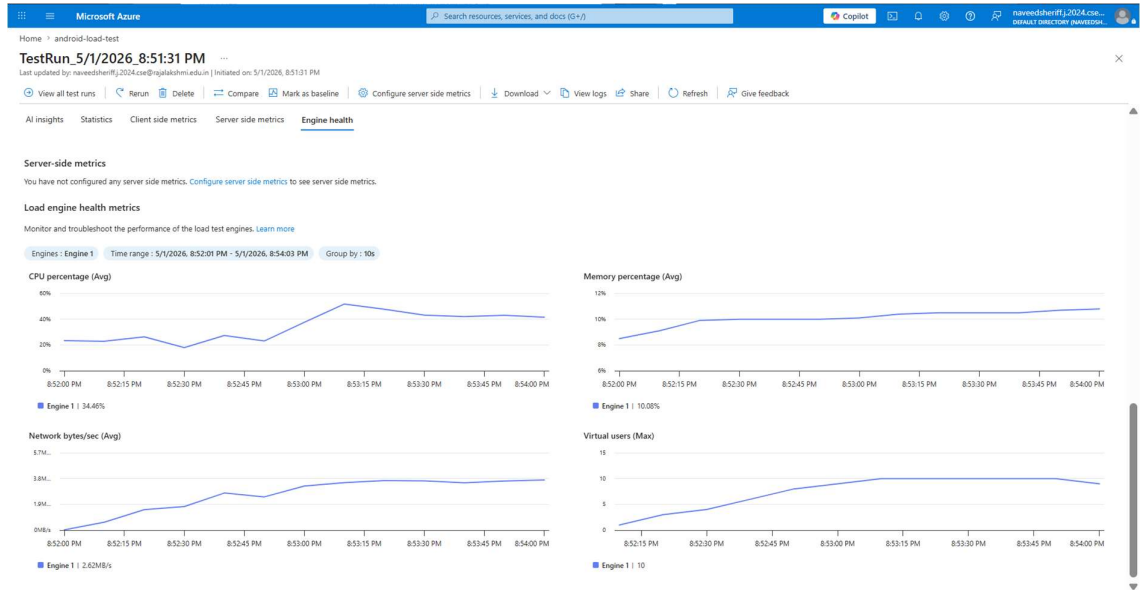
Aim:

To perform load testing on an Android application backend using Azure Load Testing by sending multiple requests to the Firebase Firestore API and evaluate system performance under load.

Procedure:

1. Open Azure Portal and sign in with valid credentials.
2. Create an Azure Load Testing resource for the project.
3. Open the created load testing resource from the dashboard.
4. Click on “Create Test” to configure a new load test.
5. Enter the test name and description for the project.
6. Add the backend API endpoint of the Speech Therapy Clinical Workflow System.
7. Select the HTTP request method such as GET or POST.
8. Configure request headers and required authentication details.
9. Set the number of virtual users for the load test.
10. Configure the test duration and ramp-up time.
11. Start the load test execution using the configured settings.
12. Monitor the response time, throughput, and error rate during execution.
13. Analyze the test results and identify system performance under load conditions.
14. Verify whether the system handles multiple requests successfully.
15. Save the load testing report and review the performance metrics.





Result:

Thus, the load test was successfully completed and the application's performance was evaluated based on response time, throughput, and error rate.